JAGAT GURU NANAK DEV PUNJAB STATE OPEN UNIVERSITY, PATIALA

DEPARTMENT OF COMPUTER SCIENCE & APPLICATIONS

M.Sc. (Computer Science) — Semester-IV

PROJECT SYNOPSIS

MSCS-4-03P: Major Project (Credits: 10 | Total Marks: 200)

Project Title:

Design and Implementation of a High-Throughput, Offline-First, Multi-Tenant SaaS Enterprise POS Architecture with Integrated AI Copilot and Computer Vision OCR

Submitted in partial fulfillment of the requirements for the degree of

Master of Science in Computer Science — M.Sc. (CS)

Semester IV (Final Semester)

```
JAGAT GURU NANAK DEV PUNJAB STATE OPEN UNIVERSITY, PATIALA
(Established by Act No. 19 of 2019 of the Legislature of State of
```

Candidate Name: `Sumit Singh`

Roll No. / Enrollment ID: `PSOU/MSCS/2026/042`

Course Code: `MSCS-4-03P`

Under the Guidance of: `Dr. Harpreet Singh (Assistant Professor, Dept of CS)`

1. INTRODUCTION

The rapid digitization of commercial retail and hospitality enterprises has necessitated modern, web-scale Point-of-Sale (POS) and Enterprise Resource Planning (ERP) systems capable of orchestrating multi-channel ordering, real-time kitchen operations, automated

inventory depletion, and unified fiscal accounting across geographically distributed franchise outlets. However, prevailing cloud-hosted architectures introduce critical architectural vulnerabilities, most notably their absolute dependency on uninterrupted public Internet connectivity, high round-trip latency during peak counter rushes, and significant security risks of cross-tenant data leakage in shared database clusters. To eliminate these vulnerabilities, this project presents the **SSR ONE AI** platform—a resilient, high-throughput, offline-first, multi-tenant SaaS architecture designed to guarantee sub-millisecond local billing latency ($< 0.5\text{ms}$) via client-side transactional IndexedDB persistence (Dexie.js), background eventual consistency synchronization via FastAPI and asynchronous PostgreSQL 16, kernel-level database isolation enforced through PostgreSQL Row-Level Security (RLS), full-duplex WebSocket real-time kitchen dispatch, and an integrated domain-specific Generative AI Copilot alongside localized Tesseract OCR for automated supplier invoice digestion.

---

2. LITERATURE SURVEY

The architecture of high-concurrency enterprise software bridges distributed database theory, client-side persistence, multi-tenant cybersecurity, and automated artificial intelligence. To contextualize the innovations in this study, the following papers, foundational techniques, and software paradigms were reviewed:

Review 1: Offline-First Architectures & Eventual Consistency in Web Applications

Citation: Kleppmann, M., Wiggins, A., van Hardenberg, M., & McGranaghan, M. (2019). *"Local-first software: you own your data, in spite of the cloud."* Proceedings of the ACM on Human-Computer Interaction, 3(OOPSLA), 1-26.

Analysis: Kleppmann et al. formalize the "local-first" computing paradigm, proving that user experience latency and operational availability improve by orders of magnitude when the primary copy of operational data resides in client-controlled local storage (e.g., IndexedDB) rather than behind a remote HTTP API. The paper demonstrates that decentralized state synchronization using Conflict-Free Replicated Data Types (CRDTs) or append-only transactional operational queues resolves network partitioning without stalling user interaction.

Relevance to Project: In SSR ONE AI, the POS cashier terminal applies this exact local-first thesis. Rather than awaiting a remote round-trip HTTP response, cashier transactions are committed to client-side IndexedDB in $< 0.5\text{ms}$ using Dexie.js, while a decoupled background worker (`useZeroWaitOrderSync`) drains pending mutations asynchronously to the PostgreSQL server when connectivity is healthy.

Review 2: Multi-Tenancy Isolation and Row-Level Security in Relational Databases

Citation: Curino, C., Jones, E. P., Zhang, Y., & Madden, S. (2010). *"Relational cloud: A database service for multi-tenant applications."* Sixth Biennial Conference on Innovative Data Systems Research (CIDR).

Analysis: This paper examines the trade-offs between shared-process, shared-database, and dedicated-database multi-tenant architectures. Curino et al. identify that while separate database instances provide physical isolation, they impose catastrophic resource overheads (connection pool fragmentation, memory waste, and complex schema migration lifecycles) as tenant counts scale into thousands. The study highlights that shared-database multi-tenancy backed by kernel-level Row-Level Security (RLS) offers superior hardware density while preventing cross-tenant leakage.

Relevance to Project: SSR ONE AI leverages PostgreSQL 16 Row-Level Security policies (`ENABLE ROW LEVEL SECURITY`) coupled with runtime session variables (`SET LOCAL app.current_tenant_id = '...'`). Every operational table inherits from `TenantBaseModel`, ensuring that isolation is enforced at the database engine level rather than relying on error-prone application-layer `WHERE` clauses.

Review 3: Asynchronous Event-Driven Architectures (ASGI vs. WSGI)

Citation: Fowler, M. (2014). *"Microservices: a definition of this new architectural term."* IEEE Software, 31(3), 87-92; and Uvicorn/FastAPI Engineering Specifications (2023).

Analysis: Traditional Python web architectures relied on the Web Server Gateway Interface (WSGI, e.g., Django, Flask), which binds each incoming HTTP request to a dedicated synchronous operating system thread. Under high I/O concurrency (e.g., multiple cashiers, QR code diners, and kitchen display polling), thread pools exhaust rapidly, leading to memory bloat and connection timeouts. The Asynchronous Server Gateway Interface (ASGI), built upon Python's `asyncio` event loop, decouples socket management from thread allocation, allowing a single worker process to handle tens of thousands of concurrent non-blocking connections.

Relevance to Project: SSR ONE AI's backend is implemented using FastAPI and Uvicorn. Non-blocking asynchronous database drivers (`asyncpg`, SQLAlchemy 2.0 AsyncIO) ensure that long-running operations (such as generating daily analytical reports or awaiting external payment gateway responses) never stall the event loop for active counter transactions.

Review 4: Domain-Specific Generative AI Copilots in Enterprise Decision Support

Citation: Brown, T., Mann, B., Ryder, N., et al. (2020). *"Language Models are Few-Shot Learners."* Advances in Neural Information Processing Systems (NeurIPS), 33, 1877-1901; and Gao, Y., et al. (2023). *"Retrieval-Augmented Generation for Large Language Models: A Survey."* arXiv:2312.10997.

Analysis: While general-purpose Large Language Models (LLMs) demonstrate vast linguistic capabilities, they suffer from factual hallucination and lack contextual awareness of real-time enterprise operational data. Retrieval-Augmented Generation (RAG) and structured persona specialization constrain LLM outputs by injecting real-time database metrics into dynamically formatted system prompts, transforming conversational models into auditable business intelligence agents.

Relevance to Project: Our platform integrates a multi-agent Copilot featuring five distinct system personas (`CEO` , `Kitchen` , `CRM` , `Finance` , `General`). Prompts are dynamically bound with tenant-scoped sales and inventory metrics, and responses are delivered via low-latency Server-Sent Events (SSE) streaming (`text/event-stream`), with complete audit trail persistence in PostgreSQL.

Review 5: Computer Vision Optical Character Recognition for Commercial Invoices

Citation: Smith, R. (2007). *"An Overview of the Tesseract OCR Engine."* Ninth International Conference on Document Analysis and Recognition (ICDAR), IEEE, 2, 629-633.

Analysis: Smith outlines the multi-stage architecture of open-source Tesseract OCR: adaptive binarization, connected component analysis, line finding, baseline fitting, and two-pass word recognition. Commercial paper receipts and invoices present unique challenges due to varied thermal typography, skewed scans, and variable table formatting, requiring adaptive thresholding and morphological preprocessing prior to character classification.

Relevance to Project: SSR ONE AI embeds localized Tesseract OCR and OpenCV image preprocessing within containerized backend workers. When kitchen staff photograph supplier paper bills, the system cleans image noise, extracts tabular line items and GST tax totals, and pre-populates draft inventory purchase orders without recurring third-party cloud API costs.

---

3. RESEARCH GAPS

Despite significant advancements in enterprise software, existing commercial and open-source solutions exhibit three critical research and engineering gaps:

The Fragility of Cloud-Bound Latency: Existing cloud POS web applications rely on continuous synchronous client-server round-trips. When localized Wi-Fi or cellular connections drop during high-pressure dining rushes, the user interface locks, causing double-billing collisions and counter paralysis. Conversely, legacy offline POS systems run isolated desktop databases that fail to synchronize real-time kitchen screens across multi-terminal setups.

Application-Layer Tenancy Vulnerabilities: The majority of multi-tenant web systems enforce tenant segregation purely via application software logic (e.g., ORM

query filters). If a developer omits a `tenant_id` filter in a single API endpoint, cross-tenant data leaks occur.

Absence of Unified Multimodal AI in Core ERP: Existing retail ERPs treat artificial intelligence and document digitization as fragmented, expensive third-party external add-ons, lacking unified, real-time, tenant-isolated conversational copilots and zero-cost local OCR parsing.

---

4. PROBLEM FORMULATION

Formally, let the enterprise platform be represented as a distributed multi-tenant system $\mathcal{S} = \langle \mathcal{T}, \mathcal{B}, \mathcal{C}, \mathcal{K}, \mathcal{D} \rangle$, where $\mathcal{T} = {t_1, t_2, \dots, t_n}$ denotes the set of isolated tenants, $\mathcal{B}$ denotes physical branch nodes, $\mathcal{C}$ denotes cashier client terminals, $\mathcal{K}$ denotes Kitchen Display System (KDS) screens, and $\mathcal{D}$ represents the relational database cluster. Each transaction generated at terminal $c \in \mathcal{C}$ at timestamp τ produces an order state mutation $m = \langle \text{idempotency_key}, t_i, b_j, \text{items}, \text{total} \rangle$. The core problem is to design an architectural state machine and data synchronization protocol that satisfies three simultaneous invariants:

Zero-Latency Invariant: The local terminal commit time must satisfy $T_{\text{local}}(m) < 1\text{ms}$, ensuring the cashier UI is immediately unlocked regardless of network state $N(\tau) \in {\text{Connected}, \text{Degraded}, \text{Disconnected}}$.

Strict Multi-Tenant Isolation Invariant: For any query q executed by user u belonging to tenant t_a, the accessible data manifold $\mathcal{M}(q)$ must satisfy: $$\mathcal{M}(q) \cap {d \in \mathcal{D} \mid d.\text{tenant_id} \neq t_a} = \emptyset$$ enforced mathematically at the database kernel via PostgreSQL Row-Level Security.

Global Eventual Consistency Invariant: For all mutations buffered offline during network disruption $\Delta \tau$, upon network restoration, the background synchronization engine must guarantee that all remote states converge to identical values without duplicate order token collisions: $$\lim_{t \to \infty} |\mathcal{S}_{\text{local}} - \mathcal{S}_{\text{server}}| = 0$$

---

5. OBJECTIVES OF THE PROJECT

The **major aim** of this project is to architect, develop, validate, and deploy a high-throughput, offline-first, multi-tenant cloud-native SaaS Enterprise Point-of-Sale (POS) and ERP platform with integrated artificial intelligence and computer vision.

To achieve this major aim, the following **specific technical objectives** are proposed:

Objective 1 — Offline-First Zero-Wait Persistence: Implement an in-memory Zustand reactive cart and client-side IndexedDB database (via Dexie.js) that commits transactions locally in $< 0.5\text{ms}$ with RFC4122 UUIDv4 idempotency keys, decoupling UI responsiveness from remote network latency.

Objective 2 — High-Throughput Asynchronous Backend: Develop an asynchronous micro-framework backend utilizing Python 3.11, FastAPI (ASGI event loop), and SQLAlchemy 2.0 AsyncIO with PgBouncer connection pooling to support high concurrent transaction throughput.

Objective 3 — Kernel-Enforced Multi-Tenant Security: Enforce PostgreSQL 16 Row-Level Security (RLS) policies and stateless JSON Web Token (JWT) role-based access control (RBAC) to mathematically guarantee tenant data isolation across shared computational resources.

Objective 4 — Real-Time Kitchen Synchronization & Hardware Printing: Implement full-duplex WebSockets with automatic ping/pong heartbeats for sub-second KDS kitchen ticket dispatch, alongside an asynchronous ESC/POS thermal printing queue worker for 80mm/58mm paper rolls.

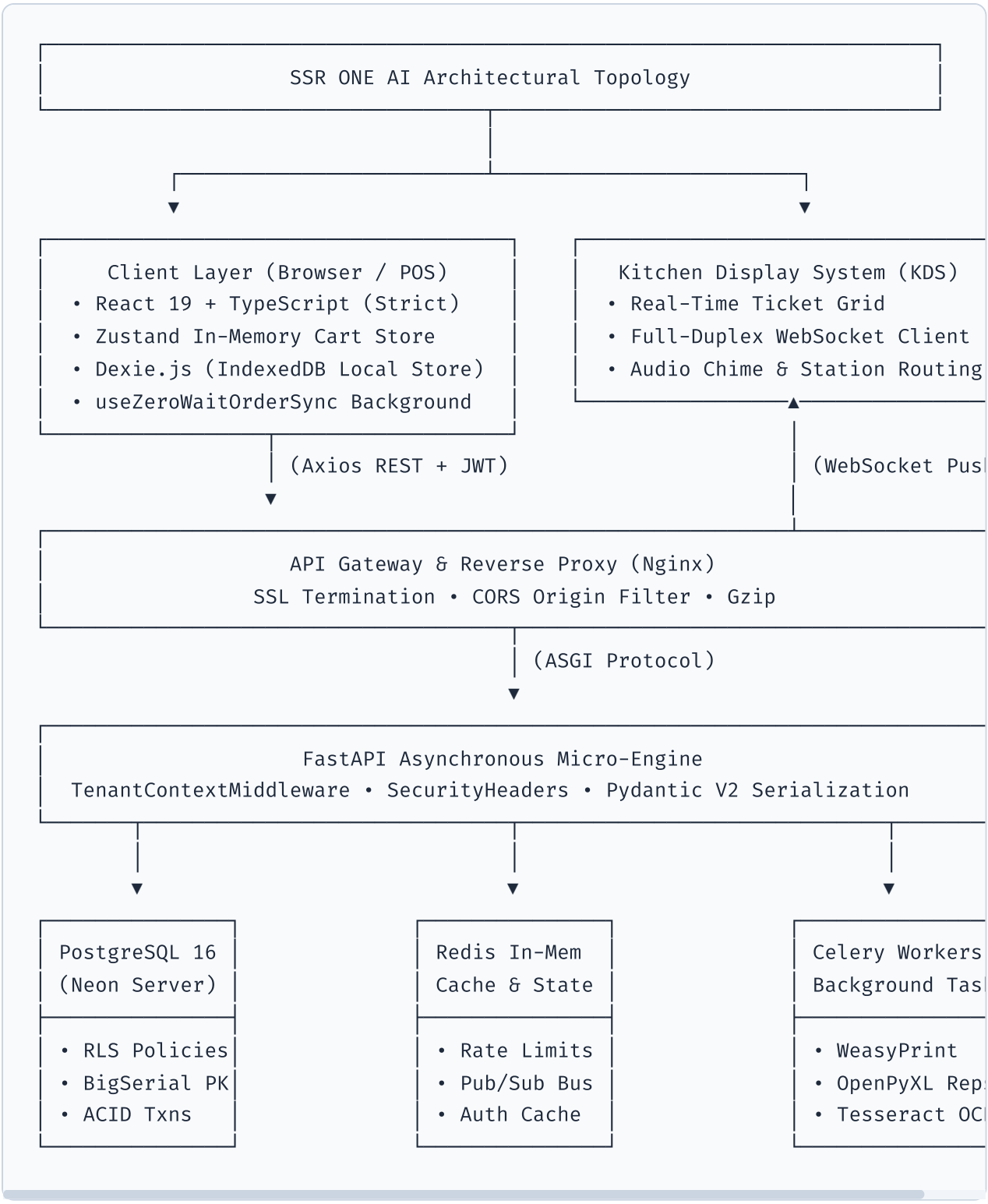
Objective 5 — Embedded AI Copilot & Computer Vision OCR Engine: Build an auditable multi-agent conversational AI Copilot (CEO, Kitchen, CRM, Finance) utilizing Server-Sent Events (SSE) streaming and integrate localized Tesseract OCR for automated supplier invoice digestion.

---

6. METHODOLOGY & PLANNING OF WORK

6.1 Research Type and Engineering Methodology

This project adopts an **Applied Systems Engineering and Experimental Architecture Methodology**, executed via an iterative Agile/Spiral Software Development Life Cycle (SDLC). The study utilizes performance benchmarking, transactional stress-testing, and penetration testing to validate architectural invariants.


SSR ONE AI Architectural Topology
Client Layer (Browser / POS)
• React 19 + TypeScript (Strict)
• Zustand In-Memory Cart Store
• Dexie.js (IndexedDB Local Store)
• useZeroWaitOrderSync Background
Kitchen Display System (KDS)
• Real-Time Ticket Grid
• Full-Duplex WebSocket Client
• Audio Chime & Station Routing
(Axios REST + JWT)
(WebSocket Pus
API Gateway & Reverse Proxy (Nginx)
SSL Termination • CORS Origin Filter • Gzip
(ASGI Protocol)
FastAPI Asynchronous Micro-Engine
TenantContextMiddleware • SecurityHeaders • Pydantic V2 Serialization
PostgreSQL 16
(Neon Server)
• RLS Policies
• BigSerial PK
• ACID Txns
Redis In-Mem
Cache & State
• Rate Limits
• Pub/Sub Bus
• Auth Cache
Celery Workers
Background Tas
• WeasyPrint
• OpenPyXL Rep
• Tesseract OC

6.2 Work Breakdown & Project Schedule (Gantt Milestones)

Phase	Milestone Description	Key Technical Activities	Timeline
I	**Requirements & Modeling**	Domain analysis of hospitality operations, ER modeling, multi-tenant hierarchy definition (`Tenant` ➡ `Company` ➡ `Branch`).	Weeks 1–2
II	**Data Layer & RLS Kernel**	PostgreSQL DDL scripting, Alembic async migration setup, RLS policy implementation, SQLAlchemy 2.0 ORM base classes.	Weeks 3–4
III	**Core Asynchronous Backend**	FastAPI ASGI route implementation, JWT auth middleware, daily order sequence generator, RESTful CRUD endpoints.	Weeks 5–6
IV	**Offline-First Frontend POS**	React 19 UI assembly, Zustand store setup, Dexie.js IndexedDB integration, `useZeroWaitOrderSync` background sync hook.	Weeks 7–8
V	**Real-Time KDS & Hardware**	WebSocket `KDSConnectionManager` implementation, thermal ESC/POS receipt generation, Celery background worker queues.	Weeks 9–10
VI	**AI Copilot & OCR Engine**	Multi-agent prompt engineering, SSE streaming endpoints, Tesseract OCR image preprocessing pipeline.	Weeks 11–12
VII	**Benchmarking & Final Thesis**	Concurrency stress-testing, latency profiling, documentation of findings, dissertation thesis writing.	Weeks 13–15

7. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

Development & Host Machine: x86-64 / ARM64 processor (Intel Core i5/i7 or Apple Silicon), minimum 16 GB DDR4/DDR5 RAM, 500 GB NVMe Solid State Drive (SSD).

Client Deployment Terminals: Commercial POS Touch Terminal, Android Tablet / iPad, or Desktop PC with standard modern web browser.

Peripheral Hardware: 80mm / 58mm ESC/POS USB/LAN Thermal Receipt Printer, Handheld 1D/2D Barcode & QR Scanner.

Software Requirements:

Operating System: Linux (Ubuntu 22.04 LTS / Debian 12) or Windows 11 with WSL2.

Runtime Environments: Node.js (v20+ LTS), Python (v3.11+).

Containerization & Orchestration: Docker Engine (v26+), Docker Compose (v2+).

Database Engines: PostgreSQL 16 (with Row-Level Security enabled), Redis 7 (In-memory cache & message broker).

Frameworks & Libraries: React 19, TypeScript 5, Vite, FastAPI, SQLAlchemy 2.0, Pydantic V2, Celery, WeasyPrint, Tesseract-OCR, Tailwind CSS.

---

8. EXPECTED OUTCOMES

The successful completion of this project will yield a production-grade, enterprise-ready hospitality POS and ERP system characterized by:

Verified Zero-Wait Performance: Cashier billing response times reduced to $< 0.5\text{ms}$ locally, completely unaffected by remote network congestion or internet